AI-POWERED FAKE NEWS AND MISINFORMATION DETECTION SYSTEM

A COMMON ASSIGNMENT REPORT

Submitted in partial fulfilment of the requirements

for the Course of

CSA1706 – ARTIFICIAL INTELLIGENCE

to the award of the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

K. Deeksha Sree (192424097)

Under the Supervision of

Dr. Krishna Kumar Kathiresan, Professor

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Common Assignment entitled "**AI-Powered Fake News and Misinformation Detection System**" has been carried out by **K. Deeksha Sree (192424097)** under the supervision of **Dr. Krishna Kumar Kathiresan, Professor,** and is submitted in partial fulfilment of the requirements for the current semester of the Bachelor of Engineering in Artificial Intelligence and Data Science programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY
Dr. Krishna Kumar Kathiresan Professor	Dr. Krishna Kumar Kathiresan Professor

STUDENT	REGISTER NUMBER	SIGNATURE
K. Deeksha Sree	192424097	_____

DECLARATION

I, **K. Deeksha Sree (192424097)**, of the Department of Artificial Intelligence and Data Science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Common Assignment Work entitled "**AI-Powered Fake News and Misinformation Detection System**" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity.

I further declare that the data, results, analysis, and findings presented in this report have not been knowingly fabricated or manipulated. All external datasets, references, software libraries, computational resources, and AI-assisted resources used for the development and documentation of this project have been appropriately acknowledged.

Place: Chennai

Date: September 2026

K. Deeksha Sree (192424097)

Signature: _____

SIMATS ENGINEERING

INDIVIDUAL CONTRIBUTION STATEMENT

As this is an individual Common Assignment, the complete project work is attributed to the student named below.

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Contribution
K. Deeksha Sree (192424097)	Problem formulation, dataset preparation, NLP preprocessing, TF-IDF feature extraction, Naive Bayes implementation, Flask integration, testing and documentation.	Designed and implemented the complete text-classification pipeline using TF-IDF and Naive Bayes.	Prepared test cases, evaluated classification metrics, reviewed errors and documented limitations.	Prepared the complete report, appendices, references and reflection.	100%

ACKNOWLEDGEMENT

I express my heartfelt gratitude to all those who supported and guided me throughout the successful completion of this Common Assignment. I am deeply thankful to the management of Saveetha Institute of Medical and Technical Sciences for providing a supportive academic environment and the facilities required for learning and project development.

I sincerely thank the faculty members of the Saveetha School of Engineering for their guidance, encouragement, and valuable academic support. I am especially grateful to my guide, **Dr. Krishna Kumar Kathiresan, Professor**, for his constructive suggestions, continuous feedback, and encouragement throughout the development of this work.

I also acknowledge the researchers, dataset contributors, and open-source communities whose publications, datasets, documentation, and software libraries supported the development of this project.

Finally, I thank my family, friends, and classmates for their encouragement and support during the completion of the assignment.

K. Deeksha Sree (192424097)

ABSTRACT

The rapid growth of social media, online news platforms, and digital communication has made it increasingly difficult to distinguish genuine news from fake or misleading information. The AI-Powered Fake News and Misinformation Detection System is developed as a text-based machine-learning application that assists users in identifying whether a given news item is likely to be REAL or FAKE.

The proposed system uses Natural Language Processing (NLP) techniques to preprocess textual news content and Term Frequency–Inverse Document Frequency (TF-IDF) to convert the processed text into numerical features. The Naive Bayes algorithm is selected as the primary classification technique because it is computationally efficient, simple to implement, and well suited to sparse text features. A labelled dataset containing 3,729 news records is used, consisting of 1,877 FAKE and 1,852 REAL records as reported in the supplied project material. An 80/20 train-test split provides a test set of 746 records.

The supplied project report reports a Naive Bayes accuracy of 96.11%, with fake-news F1-score of 96.27% and real-news F1-score of 95.94%. These values are treated as preliminary reported results and should be regenerated from the final code and exact test split before final academic submission. The trained model is integrated into a Flask-based web interface through which a user can enter news text and receive a REAL NEWS or FAKE NEWS prediction.

The system is intended as a first-level decision-support and misinformation-awareness tool rather than an absolute fact-checking service. Important information should always be independently verified using reliable sources.

Keywords: Fake News Detection, Misinformation Detection, Natural Language Processing, TF-IDF, Naive Bayes, Text Classification, Machine Learning, Flask.

TABLE OF CONTENTS

S. No.	TOPIC	SUB-SECTIONS
1	INTRODUCTION AND ENGINEERING PROBLEM	1.1 Background Information; 1.2 Need for the Project; 1.3 Problem Statement; 1.4 Project Aim; 1.5 Project Objectives; 1.6 Scope; 1.7 Expected Engineering Outcome
2	LITERATURE REVIEW AND EXISTING SOLUTIONS	2.1 Review Method; 2.2 Existing Approaches; 2.3 Naive Bayes for Text Classification; 2.4 Comparative Analysis; 2.5 Research/Engineering Gap; 2.6 Proposed Contribution
3	ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	3.1 Requirements; 3.2 Constraints & Standards; 3.3 Design Specifications; 3.4 Alternative Solutions; 3.5 Selected Design; 3.6 Trade-off Analysis; 3.7 Sustainability, Ethics, Safety, Risk & Security
4	IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	4.1 Development Approach; 4.2 Tools & Technologies; 4.3 Implementation Workflow; 4.4 Test Plan; 4.5 Results; 4.6 Engineering Analysis; 4.7 Challenges; 4.8 Project Planning & Budget
5	CONCLUSION, FUTURE WORK AND REFLECTION	5.1 Conclusion; 5.2 Future Work; 5.3 Key Learning Outcomes; 5.4 Challenges Encountered and Overcome; 5.5 Applications of Engineering Standards; 5.6 Applications of Ethical Standards; 5.7 Insight into Industry; 5.8 Personal Development
6	REFERENCES	—
7	APPENDICES	A. Algorithm; B. Source Code Structure; C. Test Evidence; D. Screenshots; E. Common Assignment Compliance

LIST OF FIGURES

Figure No.	Figure Name
Figure 1.1	Complete AI-Powered Fake News Detection System Workflow
Figure 3.1	System Architecture of the Proposed Detection System
Figure 3.2	Naive Bayes Classification Workflow
Figure 4.1	Dataset Class Distribution
Figure 4.2	Naive Bayes Model Evaluation Output
Figure 4.3	Fake News Prediction Web Interface

LIST OF TABLES

Table No.	Table Name
Table 1.1	Problem Formulation
Table 2.1	Comparative Analysis of Existing Approaches
Table 3.1	Stakeholder Requirements Matrix
Table 3.2	Functional and Non-Functional Requirements
Table 3.3	Applicable Standards and Compliance
Table 3.4	Design Alternatives Evaluation
Table 3.5	Risk Register and Mitigation Strategies
Table 4.1	Hardware and Software Tools Used
Table 4.2	Test Plan and Verification Cases
Table 4.3	Naive Bayes Performance Metrics
Table 4.4	Project Planning and Resource Allocation

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	Dimensionless
ML	Machine Learning	Dimensionless
NLP	Natural Language Processing	Dimensionless
TF-IDF	Term Frequency–Inverse Document Frequency	Dimensionless
NB	Naive Bayes	Dimensionless
TP	True Positive	Count
TN	True Negative	Count
FP	False Positive	Count
FN	False Negative	Count
API	Application Programming Interface	Dimensionless
UI	User Interface	Dimensionless
CSV	Comma-Separated Values	File format
SDG	Sustainable Development Goal	Dimensionless

CHAPTER-1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background Information

The rapid development of the internet, social media, and digital news platforms has made information available to people within seconds. While this has improved communication and access to knowledge, it has also increased the speed at which false, inaccurate, and misleading information can spread.

Fake news is commonly used to describe false or misleading information presented in a news-like format. Misinformation can be unintentionally inaccurate or deliberately misleading. The large volume of online information makes it impractical for users to manually verify every item before reading, sharing, or acting on it.

From an engineering perspective, automated text classification provides a useful first-level screening mechanism. A machine-learning model can learn statistical relationships between textual features and previously labelled REAL/FAKE examples. The proposed system combines Natural Language Processing, TF-IDF feature extraction, Naive Bayes classification, and a Flask web interface.

1.2 Need for the Project

- Misinformation can spread rapidly through websites, social media, messaging applications, and online communities.
- Manual fact-checking is time-consuming and difficult to scale to large volumes of content.
- Users require a quick first-level indication before accepting or sharing suspicious information.
- Naive Bayes provides a lightweight classification approach suitable for sparse textual data.
- The project demonstrates the practical integration of AI, NLP, machine learning, and web deployment.

1.3 Problem Statement

To design and implement an AI-powered text-classification system that accepts news content, performs NLP preprocessing and TF-IDF feature extraction, applies the Naive Bayes algorithm,

and predicts whether the supplied content belongs to the REAL or FAKE class through an easy-to-use web interface.

1.4 Project Aim

The primary aim is to develop a lightweight, interpretable, and practical fake-news and misinformation pre-screening system using Naive Bayes and NLP techniques.

1.5 Project Objectives

1. Prepare a labelled dataset containing REAL and FAKE news records.
2. Clean and normalize the news text using NLP preprocessing.
3. Convert processed text into numerical TF-IDF features.
4. Train a Naive Bayes classifier using the labelled training data.
5. Evaluate the classifier using accuracy, precision, recall, F1-score, and confusion matrix.
6. Integrate the trained model into a Flask-based prediction interface.
7. Present the prediction responsibly and clearly communicate the limitations of automated classification.

1.6 Scope

The project focuses on text-based fake-news and misinformation pre-screening.

- Included: labelled news dataset, text preprocessing, TF-IDF, Naive Bayes training, testing, prediction, evaluation, and Flask deployment.
- Excluded: complete factual verification, real-time monitoring of all social-media platforms, image/video/audio misinformation detection, and guaranteed detection of newly emerging misinformation.

1.7 Expected Engineering Outcome

The expected outcome is a reproducible software prototype that accepts news text, applies the same preprocessing and TF-IDF pipeline used during training, predicts a REAL/FAKE class using Naive Bayes, and displays the result through a Flask web interface.

Element	Formulation
Input	News headline, article text, or combined news content
Preprocessing	Lowercasing, noise/punctuation removal, tokenization and stop-word handling
Features	TF-IDF vector representation

Algorithm	Naive Bayes classifier
Output	REAL NEWS or FAKE NEWS
Evaluation	Accuracy, precision, recall, F1-score and confusion matrix
Deployment	Flask web application

Table 1.1 – Problem Formulation

CHAPTER-2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review considers established work related to fake-news detection, NLP, text classification, TF-IDF, Naive Bayes, classical machine learning, deep learning, transformer models, and the broader socio-technical problem of misinformation. The supplied reference report identifies research covering classical machine learning, BERT-based approaches, information disorder, and online information spread.

2.2 Existing Approaches

Manual fact-checking depends on human expertise and contextual judgement and can be highly effective, but it requires significant time and effort. Rule-based filtering is simple and fast but is sensitive to wording changes. Classical machine-learning approaches such as TF-IDF combined with Naive Bayes are attractive for lightweight applications because they work well with high-dimensional sparse text features.

Deep-learning and transformer-based approaches can provide richer contextual representations. However, they generally require more computational resources, more complex training, and additional deployment effort. For an academic prototype where reproducibility and efficiency are important, Naive Bayes provides a practical baseline.

2.3 Naive Bayes for Text Classification

Naive Bayes is a probabilistic supervised-learning algorithm based on Bayes' theorem. For a document represented by features X and a class C, the model estimates the posterior probability of the class given the observed features.

The classifier uses a simplifying conditional-independence assumption between features. Although natural language features are not truly independent, the assumption makes the model computationally efficient and often effective for text classification.

In the proposed system, TF-IDF provides the numerical representation of each document, while Naive Bayes learns the statistical relationship between the TF-IDF features and the REAL/FAKE labels.

2.4 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Fact-Checking	Strong contextual judgement	Slow and labour-intensive	Motivates automated pre-screening
Rule-Based Filtering	Simple and fast	Fragile and weak at context	Useful baseline
TF-IDF + Naive Bayes	Lightweight, fast, sparse-text friendly	Limited semantic understanding	Selected approach
Deep Learning / Transformers	Rich contextual representation	Higher compute and complexity	Future enhancement

Table 2.1 – Comparative Analysis of Existing Approaches

2.5 Research / Engineering Gap

A practical academic system should balance classification performance, computational cost, reproducibility, and interpretability. The proposed solution addresses this engineering need through a transparent pipeline in which text is visibly transformed into features before classification. The approach is intentionally lightweight so that the complete workflow can be trained and deployed with modest resources.

2.6 Proposed Contribution

The contribution of the project is an integrated and auditable workflow connecting dataset preparation, NLP preprocessing, TF-IDF feature extraction, Naive Bayes learning, evaluation, and web deployment. The project demonstrates how a classical probabilistic model can be transformed into a usable AI application.

CHAPTER-3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
General Public	Simple text-input interface with understandable REAL/FAKE prediction.
Students & Researchers	Transparent preprocessing, algorithm, evaluation metrics, and reproducible structure.
Project Developer	Ability to load a labelled dataset and retrain the classifier.
Academic Evaluators	Clear evidence of problem definition, design, testing, results, and limitations.

Functional and Non-Functional Requirements

Requirement	Type	Measurable Target
Accept news article text	Functional	Validate non-empty input.
Preprocess text	Functional	Apply the same preprocessing used during training.
Generate TF-IDF vectors	Functional	Transform input into model-compatible sparse features.
Predict REAL/FAKE	Functional	Return a readable class label.
Evaluate model	Functional	Calculate accuracy, precision, recall and F1-score.
Usability	Non-Functional	Readable input, result and disclaimer.
Maintainability	Non-Functional	Separate training, model, vectorizer and Flask components.
Security	Non-Functional	Validate input and dataset structure.

3.2 Constraints & Applicable Standards

The major constraints are dataset quality, model generalization, computational resources, user interpretation of predictions, input validation, and reproducibility. The following standards and engineering references are relevant to the project.

Standard / Reference	Purpose	Application
ISO/IEC 25010	Software product quality	Functional suitability, performance, usability, reliability, security and

		maintainability.
ISO/IEC 27001	Information security management	Responsible handling of datasets, input and application resources.
IEEE 29148	Requirements engineering	Definition of system requirements and boundaries.
IEEE 29119	Software testing	Structured planning and documentation of testing activities.

Table 3.3 – Applicable Standards and Compliance

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Dataset size	3,729 labelled news records	records	High
Class labels	REAL and FAKE	classes	High
Train/test split	80 / 20	percent	High
Feature representation	TF-IDF	features	High
Classifier	Naive Bayes	algorithm	High
Output	REAL NEWS / FAKE NEWS	label	High
Deployment	Flask web application	application	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 – Rule-Based Keyword Filtering: relies on manually defined suspicious keywords or patterns. It is easy to implement but can fail when the same claim is expressed using different language.

Alternative 2 – Deep Learning: models such as LSTM, Bi-LSTM, or transformer architectures can capture richer patterns but require more training effort and computational resources.

Alternative 3 – TF-IDF with Supervised Classification: converts text into sparse numerical features and applies an efficient classifier. Naive Bayes is selected for the present assignment.

Criterion	Rules	Deep Learning	TF-IDF + Naive Bayes
Implementation Complexity	Low	High	Low
Compute Requirement	Very Low	High	Low
Sparse Text Suitability	Medium	Medium	High
Context Understanding	Low	High	Low/Medium
Interpretability	High	Low/Medium	Medium/High
Assignment Suitability	Medium	Medium	High

3.5 Selected Design & Detailed Design

The selected architecture follows the pipeline: Raw News Text → Text Preprocessing → TF-IDF Feature Extraction → Naive Bayes Classifier → REAL/FAKE Prediction → Result Display.

Figure 3.1 – System Architecture of the Proposed Detection System

3.6 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
Feature representation	TF-IDF vs dense embeddings	Fast and reproducible	Limited semantic context	TF-IDF selected
Model family	Naive Bayes vs deep learning	Lower compute and simpler deployment	Less contextual understanding	Naive Bayes selected
Deployment	Local Flask vs cloud stack	Simple academic demonstration	Limited scalability	Flask selected
Evaluation	Accuracy only vs multiple metrics	Multiple metrics reveal class-wise performance	More analysis required	Use accuracy, precision, recall, F1 and confusion matrix

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Impact / Mitigation
Sustainability	Low computational footprint	Naive Bayes requires relatively modest resources.
Ethics	Dataset and prediction bias	Use representative data and avoid treating predictions as absolute truth.
Safety	User over-reliance	Display a clear disclaimer and encourage independent verification.
Security	Unsafe or malformed input	Validate text and dataset structure.
Privacy	Unnecessary storage of user text	Avoid storing user-entered news unless required.
Reliability	Unseen misinformation	Retrain periodically using diverse and recent data.

Risk Register

Risk	Likelihood	Severity	Mitigation	Residual Risk
Dataset noise / incorrect labels	Medium	High	Clean and validate records.	Low
Overfitting / weak generalization	Medium	Medium	Use held-out test data and unseen-data validation.	Low/Medium

Unseen misinformation patterns	High	High	Use diverse recent data and periodic retraining.	Medium
Misinterpretation of prediction	Medium	High	Show disclaimer and recommend verification.	Medium
Input/file errors	Medium	Medium	Validate inputs and dataset structure.	Low

3.8 Naive Bayes Algorithm

8. Load the labelled news dataset.
9. Select the text field and target label.
10. Clean and normalize the text.
11. Split the dataset into training and testing data.
12. Fit the TF-IDF vectorizer using training text.
13. Transform training and testing text into TF-IDF vectors.
14. Train the Naive Bayes classifier.
15. Predict labels for the test vectors.
16. Calculate accuracy, precision, recall, F1-score and confusion matrix.
17. Save the trained model and TF-IDF vectorizer.
18. For new text, apply the same preprocessing and vectorization.
19. Pass the resulting feature vector to Naive Bayes and display the prediction.

3.9 Pseudocode

```
BEGIN
    LOAD labelled news dataset
    CLEAN and NORMALIZE news text
    SPLIT data into training and testing sets
    FIT TF-IDF on training text
    TRANSFORM training and testing text
    TRAIN Naive Bayes classifier
    PREDICT test labels
    CALCULATE evaluation metrics
    SAVE model and vectorizer

    RECEIVE new news text
    VALIDATE input
    APPLY the same preprocessing
    TRANSFORM using saved TF-IDF vectorizer
```

PREDICT using saved Naive Bayes model
DISPLAY REAL NEWS or FAKE NEWS
DISPLAY responsible-use disclaimer
END

CHAPTER-4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system follows an iterative machine-learning and software-development workflow. Dataset inspection is followed by preprocessing, feature extraction, model training, evaluation, and Flask integration. Each stage is kept modular so that the model can be retrained without changing the user interface.

4.2 Hardware / Software / Modern Tools Used

Tool / Technology	Purpose	Stage Used
Python 3	Core programming language	Entire development
Pandas	Dataset loading and cleaning	Data preparation
NumPy	Numerical operations	Data processing
Scikit-learn	TF-IDF, Naive Bayes and metrics	Model development
NLTK	Tokenization and stop-word processing when required	NLP preprocessing
Flask	Web application and prediction interface	Deployment
HTML / CSS	User interface	Web development
CSV Dataset	Labelled news storage	Training/testing
Joblib / Pickle	Model persistence	Deployment
VS Code / Jupyter / Google Colab	Development and debugging	Development/testing

Table 4.1 – Hardware and Software Tools Used

4.3 Implementation Workflow

20. Load the CSV dataset using Pandas.
21. Inspect the text and label columns and remove unusable records.
22. Apply text normalization and preprocessing.
23. Split the data into training and testing sets using an 80/20 split.
24. Fit TF-IDF using the training text and transform both partitions.
25. Train the Naive Bayes classifier.
26. Generate test predictions.
27. Calculate evaluation metrics.

28. Save the model and vectorizer.
29. Load the saved artifacts into the Flask application.
30. Accept new user text and display the REAL/FAKE prediction.

4.4 Project Folder Structure

```
Fake-News-Misinformation-Detection/
├── Dataset/
│   └── news_dataset.csv
├── static/
│   └── style.css
├── templates/
│   └── index.html
├── app.py
├── train_model.py
├── fake_news_model.pkl
└── tfidf_vectorizer.pkl
```

4.5 Test Plan & Verification

Test Case	Test Method	Expected Result	Status
TC01	Enter valid REAL news text	System returns REAL NEWS	Verify with final run
TC02	Enter valid FAKE news text	System returns FAKE NEWS	Verify with final run
TC03	Submit empty input	Validation warning is displayed	Pass
TC04	Enter punctuation/special characters	Input is safely processed	Pass
TC05	Use uppercase/lowercase variations	Text is normalized	Pass
TC06	Load labelled CSV dataset	Records and labels are available	Pass
TC07	Retrain Naive Bayes model	Model and metrics are updated	Verify with final run
TC08	Use very short input	System handles input according to validation policy	Verify

Table 4.2 – Test Plan and Verification Cases

4.6 Results

The supplied project report provides preliminary Naive Bayes results using an 80/20 split of 3,729 records. The resulting test set contains 746 records. The reported Naive Bayes accuracy is 96.11%. The supplied report also reports fake-news F1-score of 96.27%, real-news F1-score of 95.94%, precision of 92.88%, and recall of 100.00%.

Metric	Naive Bayes – Reported Result
Accuracy	96.11%
F1-score – Fake	96.27%
F1-score – Real	95.94%
Precision	92.88%
Recall	100.00%
Test Size	746

Table 4.3 – Naive Bayes Performance Metrics

Important validation note: the values above are reproduced from the supplied project material. The final submission should regenerate all metrics from the final training code, saved test split, and final model so that the accuracy, classification report, and confusion matrix are internally consistent.

Figure 4.1 – Dataset Class Distribution

Figure 4.2 – Naive Bayes Model Evaluation Output

Figure 4.3 – Fake News Prediction Web Interface

4.7 Engineering Analysis

The reported 96.11% accuracy demonstrates that Naive Bayes provides a strong baseline for the selected dataset. Its major advantages are fast training, low computational requirements, simple implementation, and compatibility with sparse TF-IDF representations.

However, accuracy alone should not be interpreted as factual truth. The model learns statistical patterns from the training data. It does not independently verify claims, understand every contextual relationship, or guarantee correct predictions for newly emerging misinformation. Class-wise precision, recall, F1-score and confusion-matrix analysis are therefore important.

4.8 Challenges Encountered

- Preparing textual data so that inconsistent capitalization, punctuation and noise do not affect feature extraction.

- Maintaining the same preprocessing and TF-IDF transformation during both training and prediction.
- Understanding the difference between classification confidence and factual verification.
- Interpreting multiple evaluation metrics rather than depending only on accuracy.
- Integrating the trained model with a Flask web application.

4.9 Strengths and Limitations

Strengths

- Lightweight and computationally efficient.
- Suitable for sparse TF-IDF text features.
- Easy to train, evaluate and deploy.
- Clear end-to-end pipeline.
- Suitable for an academic prototype.

Limitations

- Text-only classification cannot independently verify facts.
- Performance depends on dataset quality and representativeness.
- Sarcasm, satire and ambiguous language may be difficult to classify.
- New misinformation patterns may not resemble training examples.
- Final metrics must be regenerated from the final code before submission.

4.10 Project Planning & Resource Allocation

Milestone	Planned Activity	Output
Week 1	Problem definition and requirements	Problem statement, aim and scope
Week 2	Dataset collection and inspection	Prepared labelled dataset
Week 3	NLP preprocessing and TF-IDF	Feature pipeline
Week 4	Naive Bayes training and evaluation	Model and metrics
Week 5	Flask integration and UI	Working prediction interface
Week 6	Testing and documentation	Validated prototype and report
Item	**Estimated Cost (INR)**	**Actual Cost (INR)**
Python libraries / software	0	0
Development tools	0	0
Dataset / academic resources	0	0
Local testing / internet usage	500	500
Printing / documentation	500	500
Total	1,000	1,000

Table 4.4 – Project Planning and Resource Allocation

CHAPTER-5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The AI-Powered Fake News and Misinformation Detection System demonstrates the practical use of Artificial Intelligence, Natural Language Processing, TF-IDF feature extraction, and Naive Bayes classification for text-based misinformation pre-screening. The system converts raw news text into numerical features, learns from labelled examples, evaluates classification performance, and presents a readable result through a Flask web interface.

The supplied project material reports 96.11% accuracy for Naive Bayes on a 746-record test set derived from 3,729 records. This indicates that the selected approach can provide a strong baseline on the supplied dataset. Nevertheless, the classifier should be treated as a decision-support tool rather than a factual authority.

5.2 Future Work

- Use larger, more diverse and regularly updated datasets.
- Add multilingual fake-news and misinformation detection.
- Compare Naive Bayes with Logistic Regression, SVM, BERT and other contextual models.
- Integrate trusted fact-checking sources and verified news databases.
- Add explainable AI features to show influential textual characteristics.
- Extend the system to image, audio and video misinformation.
- Introduce periodic retraining to address concept drift.
- Evaluate the system on completely unseen current news from multiple sources.

5.3 Key Learning Outcomes

- Understanding the complete NLP text-classification pipeline.
- Practical use of TF-IDF for text representation.
- Understanding Naive Bayes as a probabilistic classifier.
- Training and evaluating a supervised machine-learning model.
- Interpreting accuracy, precision, recall, F1-score and confusion matrices.
- Integrating a trained model into a Flask web application.
- Understanding responsible AI, dataset bias, limitations and human verification.

5.4 Challenges Encountered and Overcome

One major challenge was converting unstructured news text into a representation that a machine-learning model could process. This was addressed through systematic preprocessing and TF-IDF feature extraction. Another challenge was maintaining consistency between training and prediction. The same preprocessing and vectorization pipeline must be applied to new input; otherwise, the model may receive incompatible features.

A further challenge was understanding that a high classification score does not prove that a news statement is factually true. This led to the inclusion of responsible-use language and a clear distinction between classification and fact-checking.

5.5 Applications of Engineering Standards

- ISO/IEC 25010 concepts support consideration of usability, reliability, performance efficiency, security and maintainability.
- ISO/IEC 27001 principles support responsible handling of datasets, user input and application resources.
- IEEE 29148 provides a reference for defining requirements and system boundaries.
- IEEE 29119 provides a reference for structured test planning and documentation.

5.6 Applications of Ethical Standards

Ethical considerations are important because a fake-news detector can influence what users believe or share. The system should therefore communicate its limitations, avoid presenting predictions as absolute truth, and encourage users to verify important claims using reliable sources. Dataset bias should also be monitored because an imbalanced or source-specific dataset can produce misleading predictions.

5.7 Insight into Industry

The project provides insight into the growing use of machine learning for content moderation, information quality, cybersecurity, media analytics, and digital trust. In industry, a production-grade misinformation detection system would generally require larger datasets, continuous monitoring, explainability, model governance, source verification, security controls, and regular evaluation against changing information patterns.

5.8 Individual Reflection – K. Deeksha Sree

Working on the AI-Powered Fake News and Misinformation Detection System helped me understand how theoretical machine-learning concepts can be converted into a practical application. I learned that building a classification system is not limited to selecting an algorithm. Dataset preparation, text preprocessing, feature extraction, training, testing, evaluation, deployment, and documentation are all connected parts of the engineering process.

The most important technical learning from this assignment was the use of TF-IDF with Naive Bayes. Before implementing the project, textual information appeared to be difficult for a machine to process directly. By using preprocessing and TF-IDF, I understood how words can be transformed into numerical features that a classifier can use. Naive Bayes helped me understand probabilistic classification and why a simple model can still perform effectively on text data.

I also learned the importance of evaluating a model using more than accuracy. Precision, recall, F1-score and the confusion matrix provide additional information about the types of mistakes made by a classifier. This was particularly important for the project because both false acceptance of fake information and false rejection of genuine information can affect users.

Another important lesson was responsible AI. A model trained on historical examples cannot automatically verify every fact in a new article. It identifies patterns learned from the training data. Therefore, the result should be communicated as an indication rather than a guaranteed truth. This understanding influenced the design of the user interface and the disclaimer included in the project.

The assignment also improved my practical skills in Python, data handling, machine learning, debugging, web integration, testing, technical documentation, and project planning. If I continue this project, I would use a larger and more diverse dataset, evaluate the model on completely unseen current news, add explainability, and compare the Naive Bayes model with modern transformer-based approaches.

Overall, this assignment strengthened my confidence in applying artificial intelligence and data-science concepts to a real-world problem. It also taught me that good engineering requires not only performance, but also transparency, responsible interpretation, testing, and continuous improvement.

5.9 SDG Relevance

The project has a broader connection with Sustainable Development Goal 16: Peace, Justice and Strong Institutions, particularly the broader objective of supporting informed and responsible access to information. The proposed system does not eliminate misinformation, but it demonstrates a technical approach that can assist users with first-level screening and awareness.

REFERENCES

[1] Bhardwaj, M., & Rani, S. (2021). A comprehensive survey on fake news detection using machine learning and deep learning approaches. International Journal of Information Management Data Insights.

[2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT.

[3] Jurafsky, D., & Martin, J. H. Speech and Language Processing. Stanford University.

[4] Kaliyar, R. K., Goswami, A., & Narang, P. (2021). FakeBERT: Fake news detection in social media with a BERT-based deep learning approach. Multimedia Tools and Applications.

[5] Khan, J. Y., Khondaker, M. T. I., Afroz, S., Uddin, G., & Iqbal, A. (2021). A benchmark study of machine learning models for online fake news detection. Machine Learning with Applications.

[6] Lazer, D. M. J., et al. (2018). The science of fake news. Science, 359(6380), 1094–1096.

[7] McKinney, W. (2022). Python for Data Analysis. O'Reilly Media.

[8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[9] UNESCO. (2018). Journalism, Fake News & Disinformation: Handbook for Journalism Education and Training.

[10] Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. Science, 359(6380), 1146–1151.

[11] Zhou, X., & Zafarani, R. (2020). A survey of fake news: Fundamental theories, detection methods, and opportunities. ACM Computing Surveys, 53(5).

[12] Python Software Foundation. Python Documentation.

[13] Scikit-learn Developers. Scikit-learn Documentation.

[14] Flask Documentation. Flask Web Framework Documentation.

[15] International Organization for Standardization. ISO/IEC 25010: Systems and software engineering – Systems and software quality requirements and evaluation.

[16] International Organization for Standardization. ISO/IEC 27001: Information security management systems.

APPENDICES

Appendix A – Naive Bayes Algorithm

INPUT:

Labelled news dataset D

PROCESS:

1. Clean and normalize each news document.
2. Split D into training and testing sets.
3. Fit TF-IDF vectorizer on training documents.
4. Transform training and testing documents.
5. Train Naive Bayes classifier.
6. Predict labels for test documents.
7. Calculate evaluation metrics.

PREDICTION:

1. Receive new news text.
2. Preprocess text using the training pipeline.
3. Transform text using the saved TF-IDF vectorizer.
4. Predict REAL or FAKE using saved Naive Bayes model.
5. Display result and disclaimer.

Appendix B – Source Code Structure

```
Fake-News-Misinformation-Detection/
│
├── Dataset/
│   └── news_dataset.csv
│
├── static/
│   └── style.css
│
├── templates/
│   └── index.html
│
├── train_model.py
├── app.py
├── fake_news_model.pkl
└── tfidf_vectorizer.pkl
```

Appendix C – Test Evidence

Evidence No.	Evidence to Insert	Purpose
E1	Dataset loading screenshot	Shows dataset is loaded correctly.
E2	Preprocessing / TF-IDF screenshot	Shows text transformation.
E3	Naive Bayes training output	Shows model training.
E4	Classification report / confusion matrix	Shows model evaluation.
E5	REAL NEWS prediction screenshot	Shows correct interface output.
E6	FAKE NEWS prediction screenshot	Shows correct interface output.

Appendix D – Execution Screenshot Placeholders

Screenshot 1 – Dataset Loading

Screenshot 2 – Text Preprocessing

Screenshot 3 – Naive Bayes Training

Screenshot 4 – Confusion Matrix

Screenshot 5 – REAL NEWS Prediction

Screenshot 6 – FAKE NEWS Prediction

Appendix E – Common Assignment Compliance

Common Assignment Requirement	Report Location
Problem statement and formulation	Chapter 1.3 and 1.7
Objectives and expected outcome	Chapter 1.4–1.7
Stakeholder and system requirements	Chapter 3.1
Constraints and applicable standards	Chapter 3.2
Design specifications	Chapter 3.3
Alternative solutions and trade-offs	Chapter 3.4–3.6
Sustainability, ethics, safety, risk and security	Chapter 3.7
Algorithm and pseudocode	Chapter 3.8–3.9; Appendix A
Implementation and tools	Chapter 4.1–4.4
Test plan and verification	Chapter 4.5; Appendix C
Results and analysis	Chapter 4.6–4.7
Challenges and limitations	Chapter 4.8–4.9
Project planning and budget	Chapter 4.10
Conclusion and future work	Chapter 5.1–5.2
Learning outcomes and reflection	Chapter 5.3–5.8
SDG / broader consideration	Chapter 5.9
References	References
Execution screenshots	Appendix D